

eStation IoT 设备开发人员手册

文档版本: V1.7.3

示例代码

项目代码

本文档仅适用于 ETAG PTL(灯条) 系统集成。

技术支持: 联系提供商

目录

1	介绍.....	3
1.1	背景.....	3
1.2	eStation 的系统结构.....	4
2	使用 eStation.....	4
2.1	连接 eStation.....	5
2.2	接收 eStation 结果.....	6
2.3	接收 eStation 心跳.....	9
2.4	发布 PTL(灯条)任务.....	10
2.5	发布分组/绑定任务.....	11
2.6	发布群控信息.....	12
2.7	发布基站 OTA 任务（该功能仅在固件 1.5.0 以后版本支持）.....	13
3	参考.....	13
3.1	数据定义（Java 版本）.....	13
3.2	数据定义（C#版本）.....	18
3.3	电量描述.....	22
3.4	颜色描述.....	22

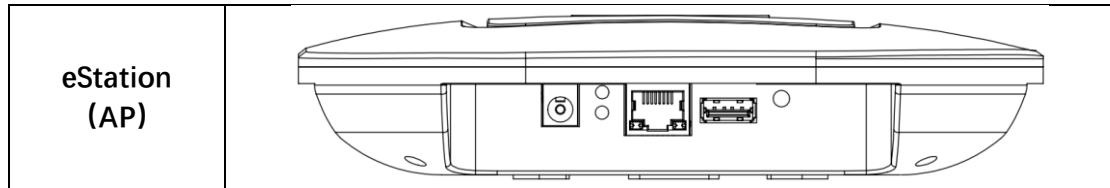
1 介绍

1.1 背景

注意：本文档文适用于行业客户定制版本的 eStation IoT 设备。

eStation IoT 设备，用于快速开发集成商的业务项目，使用 JSON 数据格式的 MQTT 协议。

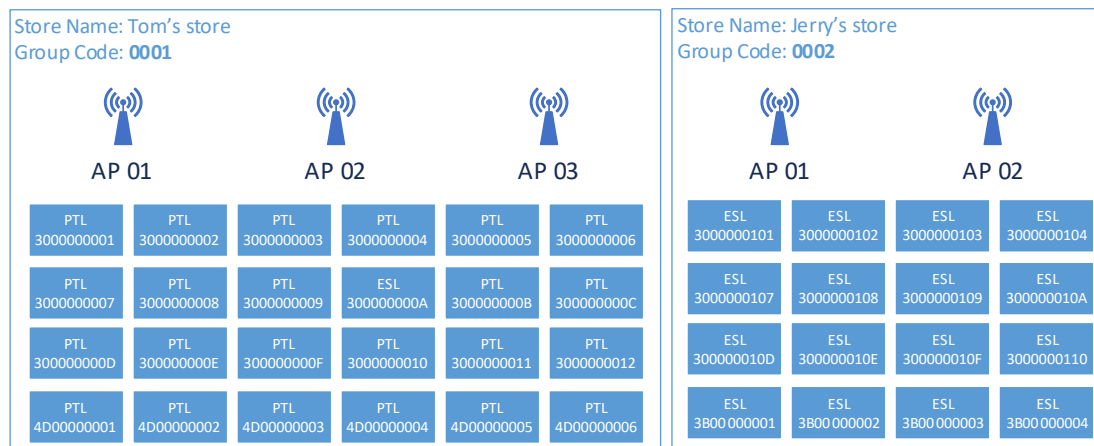
eStation 的外观如下：



在你开始使用 eStation 前, 你应该了解以下关键点:

- Group:** 如果你在一个场景里中有多个 AP, 并且你有多个场景。如果一个 PTL (灯条)沟通失败, 你可能需要调用场景里的所有 AP, 尝试与失败的 PTL (灯条)沟通。在这种情况下, 你应该将这些 AP 放在一个存储中。你应该在应用程序中管理 AP 和 Group 的关系。
- AP:** 与 PTL (灯条)相连的射频接入点。
- AP ID:** AP 和 eStation 的标识始终使用 MAC 作为其标识。
- PTL (灯条):** 灯光拣选标签。
- MQTT:** eStation (AP) 使用 JSON 格式的数据 MQTT 通信协议。

例如, 你的项目有两个场景, 像这样:



数据结构应该是:

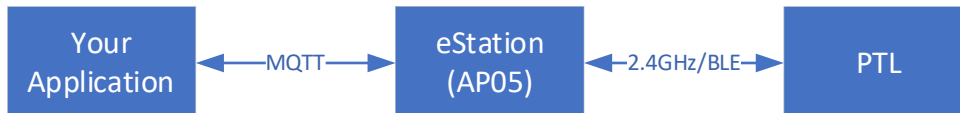
Group	AP ID	PTL (灯条) ID
0001	01	30000001, 30000002, 30000007, 30000008, 3000000D ...
	02	30000003, 30000004, 30000009, 3000000A, 3000000F ...
	03	30000005, 30000006, 3000000B, 3000000C, 30000011 ...
0002	01	30000101, 30000102, 30000107, 30000108, 3000010D ...
	02	30000103, 30000104, 30000109, 3000010A, 3000010F ...

注意: 通常, 你的应用程序应该记住与每一个 PTL(灯条)成功通信的 AP, 但物理

上, PTL(灯条)不存在与任何 AP 绑定关系。PTL(灯条)只是逻辑上尝试记住成功向其发送数据的最后一个默认 AP 的 ID。

1.2 eStation 的系统结构

基本上, 你的项目将包含 PTL(灯条) ID 和对应的闪灯/蜂鸣指令的任务发送到站点, eStation 将使用 2.4GHz 射频(RF)将任务数据发送到确切的 PTL(灯条), 并且 eStation 将结果返回给你的项目。换句话说, eStation 位于应用程序和 PTL(灯条)之间。



本文档将描述两个方面: 你的应用程序端和 eStation 端.

请记住, 如果你的应用程序和站点不在同一个专用网络中, 你应该添加一个强密码来保护连接, 并且添加一个 X.509 证书。

2 使用 eStation

如前所述, 不存在与 eStation 的代码级的集成。你应该管理应用程序中的连接和通信。

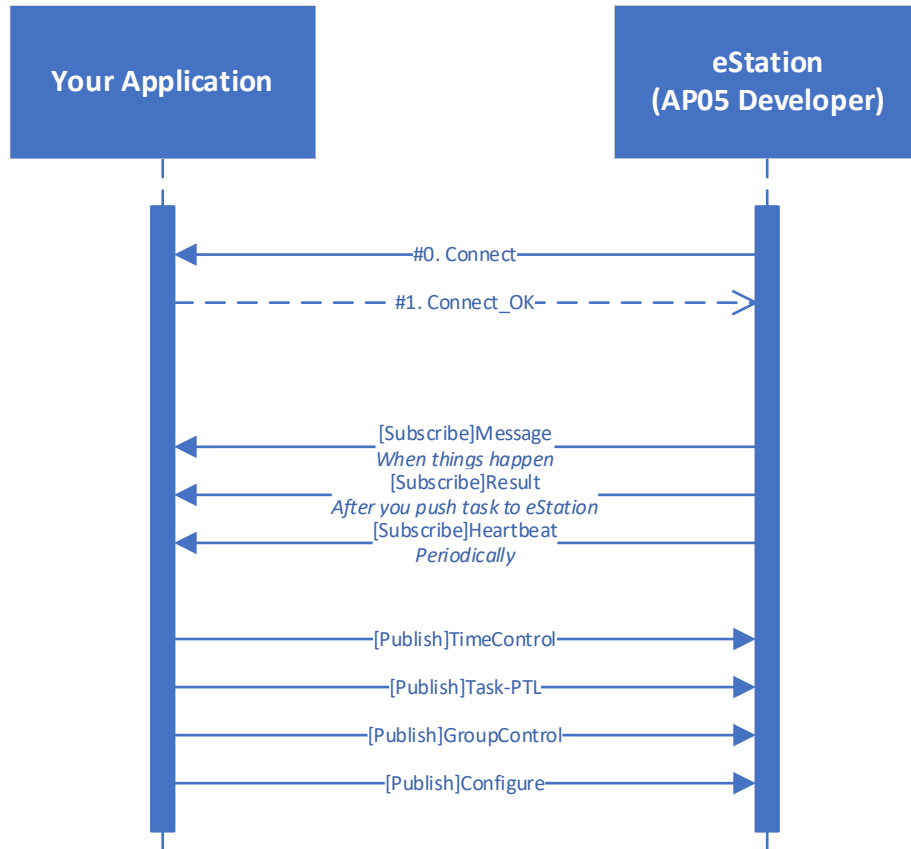
有 7 个 Topic, 3 个 Topic 从站点端发布, 4 个 Topic 从应用程序端发布。

从 eStation 端, 你的应用程序应该订阅以下 Topic:

1. **result**: eStation 将在与 PTL (灯条)通信后返回结果。
2. **heartbeat**: eStation 将定期返回当前状态。

从你的应用程序端, 站点将订阅以下 Topic:

1. **task**: 你可以发布带有此 Topic 的 PTL (灯条)任务。
2. **bind**: 你可以发布带有此 Topic 的分组功能。
3. **group**: 你可以发布带有此 Topic 的群控任务。



Note: Publish/Subscribe is from your application side

2.1 连接 eStation

eStation 作为 IoT 设备客户端工作，它将在通电后尝试连接到你的应用程序(服务端)。

eStation 的默认参数为:

默认目标服务器地址	192.168.172.172:23457 (支持 域名:端口)
默认用户名	test
默认密码	123456

注意:该用户名/密码 为默认通用密码，其他要求另外配置。

但是，如果你无法连接到 eStation，或者你忘记了它的目标服务器地址，或者你忘记了用户名/密码，你可以按重置(RESET)按钮 5 秒钟以上，它将恢复到默认的目标服务器 IP 地址，用户名和密码，然后重启即可生效。

注意:如果你重置了 eStation，但它仍然无法连接到你的应用程序，请检查你的防火墙和网线连接。**REST 功能在 1.6.2 固件版本以后支持。**

2.2 接收 eStation 结果

Topic: /estation/{ID}/result

ID: eStation 的 ID。

PayloadSegment: TaskResult 对象。TaskResult 对象包含 TaskItemResult 对象的列表。

TaskResult 属性是:

#	属性	类型	备注
0	ID	String	AP ID, AP MAC
1	TotalCount	Int	缓存中的总任务计数
2	SendCount	Int	发送任务计数 (在射频模块中)
3	Results	List<TaskItemResult>	PTL (灯条)结果列表

TaskItemResult 属性是:

#	属性	类型	备注
0	TagID	String	PTL (灯条) ID
1	Version	String	固件版本, 保留备用
2	ResultType	Int	数据返回类型, 包括: 0xFD 按键返回, 0xFE 通信返回, 0xFF 心跳返回
3	RfPowerSend	Int	AP 方发送 RF 功率, dBm, -256 表示无数据
4	RfPowerRecv	Int	灯条方环境 RF 功率, dBm, -256 表示无数据
5	Battery	Int	电池电量, V, 0 表示无数据, 需要除以 10
6	Colors	RGB[]	LED 灯的当前状态, 暂时保留字段
7	Group	Int	分组号 (基站版本 1.6.7 及以后支持)

RGB 的定义如下:

#	属性	类型	备注
0	R	Bool	红, Red
1	G	Bool	绿, Green
2	B	Bool	蓝, Blue

注意:工作站内部有一个队列, 当应用程序将任务数据发送到工作站时, 如果射频模块空闲, 它将立即发送到射频模块, 或者如果射频模块正在工作, 它将等待直到天线空闲。因此, 应用程序和 eStation 之间的通信是异步的。

注: 调测时 0xFF 灯条心跳时间是 10 分钟。基站版本 1.6.7 及以后灯条的通信消息都携带 Group 信息

```
Topic: /estation/90A9F730006E/result
// 灯条心跳消息 约10 分钟一次
Payload:
{
  "ID": "90A9F730006E",    // 基站 ID (SN)
  "TotalCount": 0,
  "SendCount": 0,
  "Results": [
    {
```

```

        "TagID": "AD1E00071E27", // 灯条 ID
        "Version": "29",        // 灯条固件版本
        "ResultType": 255,      // 0xFF 心跳返回
        "RfPowerSend": -63,
        "RfPowerRecv": -249,
        "Battery": 30,          // 电池电量, 单位是伏 (V), 除以
10 后参考文档最后一页电量对照表
        "Sequence": 7,
        "Colors": [             // 灯条当前状态, 颜色值参考文档最
后一页对照表
            {
                "R": false, // 红
                "G": false, // 绿
                "B": false  // 蓝
            }
        ],
        "Group": 0              // 灯条当前的组号
    }
]
}

// 灯条按键消息 仅按灭灯有返回
Payload:
{
    "ID": "90A9F730006E", // 基站 ID (SN)
    "TotalCount": 0,
    "SendCount": 0,
    "Results": [
        {
            "TagID": "AD1E00071E27", // 灯条 ID
            "Version": "29",        // 灯条固件版本
            "ResultType": 253,      // 0xFD 按键消息
            "RfPowerSend": -63,
            "RfPowerRecv": -249,
            "Battery": 30,          // 电池电量, 单位是伏 (V), 除以
10 后参考文档最后一页电量对照表
            "Sequence": 7,
            "Colors": [             // 灯条当前状态, 颜色值参考文档最
后一页对照表
                {
                    "R": false, // 红
                    "G": false, // 绿

```

```

        "B": false // 蓝
    }
    ],
    "Group": 0 // 灯条当前的组号
}
]
}

// 灯条通信消息 如发送亮红灯指令后, 灯条回复如下
Payload:
{
    "ID": "90A9F730006E", // 基站 ID (SN)
    "TotalCount": 0,
    "SendCount": 0,
    "Results": [
        {
            "TagID": "AD1E00071E27", // 灯条 ID
            "Version": "29", // 灯条固件版本
            "ResultType": 254, // 0xFD 按键消息
            "RfPowerSend": -63,
            "RfPowerRecv": -249,
            "Battery": 30, // 电池电量, 单位是伏 (V), 除以
10 后参考文档最后一页电量对照表
            "Sequence": 7,
            "Colors": [ // 灯条当前状态, 颜色值参考文档最
后一页对照表
                {
                    "R": true, // 红
                    "G": false, // 绿
                    "B": false // 蓝
                }
            ],
            "Group": 0 // 灯条当前的组号
        }
    ]
}

```


2.3 接收 eStation 心跳

Topic: /estation/{ID}/heartbeat

ID: eStation 的 ID.

PayloadSegment: eStationInfor 对象。

eStationInfor 属性是:

#	属性	类型	注意
0	ID	String	ID, 只读
1	MAC	String	设备 MAC 地址, 只读
2	Alias	String	设备别名, 友好视图
3	ClientType	Int	默认为 2
4	ServerAddress	端点的 String	服务器地址, 端点(IP + 端口) 默认值为:192.168.1.92:9080
5	Parameters	String 数组	连接参数、用户名和密码。
6	LocalIP	IP String	本地 IP 地址, 空表示自动从路由器获取 IP 地址。
7	SubnetMask	IP String	子网掩码
8	Gateway	String	网关
9	Heartbeat	Int	心跳速度 20 秒
10	DummyVersion	String	通信引擎版本
11	AppVersion	String	eStation 固件版本
12	TotalCount	Int	缓存中的任务总数
13	SendCount	Int	射频模块中任务的总数

```
topic: /estation/90A9F730006E/heartbeat
// 基站 90A9F730006E 心跳信息 每 20 秒一次
Payload:
{
  "ID": "90A9F730006E",
  "MAC": "90A9F730006E",
  "Alias": "",
  "ClientType": 0,
  "ServerAddress": "192.168.172.172:23457", // mqtt server
    的地址和端口
  "Parameters": [
    "test", // mqtt 账号
    "123456" // mqtt 账号密码
  ],
  "LocalIP": "192.168.172.173", // 暂未有效 请忽略
  "SubnetMask": "",
  "Gateway": "",
  "Heartbeat": "",
  "AppVersion": "1.6.7.0", // 基站程序版本
}
```

```
"TotalCount": 0,  
"SendCount": 0  
}
```

2.4 发布 PTL(灯条)任务

Topic: /estation/{ID}/task

ID: eStation 的 ID

PayloadSegment:

1. 一个 TaskData 对象。
2. 一个 TaskItemData 数组。

TaskData 属性是:

#	属性	类型	备注
0	Time	Int	取值范围: 0~255, 每一个挡位五秒 灯条的一次最大持续亮灯和蜂鸣时间 夹子灯条 (0~36) 180 秒=3 分钟 磁吸灯条 (0~255) 1275 秒≈21 分钟
1	Items	TaskItemData[]	子任务, 每一个灯条一个子任务

TaskItemData 属性是:

#	属性	类型	备注
0	TagID	String	PTL ID
1	Beep	Bool	蜂鸣器 (Beeper)
2	Colors	RGB[]	RGB 三色灯颜色编组, 定义参见 3.3 颜色描述
3	Flashing	Bool?	True: 闪烁, False: 不闪烁, Null: 灭灯

数据包长度: 需要注意的是 (单个三色灯每组最多 60 个, 超过 60 个该任务无效, 建议每组 20 个)。

RGB 三色灯: 目前支持带 1 个 RGB 三色灯。开发者需要根据实际的灯条类型传递 RGB 的灯色定义。

注: 调测时灯条 ID 需要增加前缀 AD1, 灯条标签上默认未打印前缀, 需要扫描条码显示全部。

```
发布亮灯指令  
topic: /estation/90A9F7300000/task  
Payload:  
{  
  "Items": [  
    {  
      "Beep": true,           // 蜂鸣  
      "Colors": [  
        {  
          "B": false,  
          "G": false,  
          "R": false,  
          "W": false  
        }  
      ]  
    }  
  ]  
}
```

```
        "R": true           // 红灯
    },
    ],
    "Flashing": true,       // 闪烁
    "TagID": "AD10000048F" // 灯条id
}
],
"Time": 5                  // 亮灯时间 5*5=25 秒
}
```

发布灭灯指令

topic: /estation/90A9F7300000/task

Payload:

```
{
  "Items": [
    {
      "Beep": false,
      "Colors": [
        {
          "B": false,
          "G": false,
          "R": false
        }
      ],
      "TagID": "AD10000048F"
    }
  ],
  "Time": 0
}
```

2.5 发布分组/绑定任务

Topic: /estation/{ID}/bind

ID: eStation 的 ID

PayloadSegment:

1. 一个 BindTaskData 对象，Group 为需要分组的 ID，取值范围见备注。

GroupData 属性是:

#	属性	类型	备注
1	Group	Int	取值范围: 0~254, 0 为解除, 1~254 为分组号
2	Items	String[]	灯条 ID

数据包长度: 每包上限不超过 60 个, 超过 60 个无法成功。

```
发布分组
topic: /estation/90A9F7300000/bind
Payload:
{
  "Group": 10,           // 分组号
  "Items": [
    "AD100000048F"      // 灯条列表
  ]
}
```

2.6 发布群控信息

Topic: /estation/{ID}/group

ID: eStation 的 ID

PayloadSegment:

1. 一个 GroupData 对象。

GroupData 属性是:

#	属性	类型	备注
0	Group	Int	取值范围: 0~255, 0~254 为 2.6 中定义的 分组号, 255 为全场群控
1	R	Bool	红色 LED 灯
2	G	Bool	绿色 LED 灯
3	B	Bool	蓝色 LED 灯
4	Beep	Bool	蜂鸣器 (Beeper)
5	Times	2 字节	取值范围: [0~250] LED 灯闪烁 5 * X 次, 5 * X 秒 灯条的一次最大持续亮灯和蜂鸣时间 180 秒
6	Flashing	Bool	True 为闪烁, False 为常亮 (该功能在未来固件 1.6.4 规划版本后才有)

群控指令每次发送都会执行, 连续发送间隔建议 6 秒一次。

无返回: 该群控指令不会返回任何数据。

```
发布群控
topic: /estation/90A9F7300000/group
Payload:
{
  "Group": 10,           // 组号
  "Beep": true,          // 蜂鸣
  "B": false,
}
```

```
"G": false,
"R": true,           // 红灯
"Times": 5           // 亮灯时间 5*5=25 秒
}
```

2.7 发布基站 OTA 任务（该功能仅在固件 1.5.0 以后版本支持）

Topic: /estation/{ID}/ota
ID: eStation 的 ID
PayloadSegment: 一个 OtaData 对象。
OtaData 属性是:

#	属性	类型	备注
0	Firmware	String	读取固件文件字节数组再进行 base64 编码 (Base64.NO_WRAP 无换行符)
1	Type	Int	OTA 类型, 暂定为 0
2	Version	String	固件版本, 视实际固件版本而定, 如 1.5.0
3	MD5	String	固件文件 MD5 值

无返回: 该指令不会返回任何数据, 成功后基站自动重启。

```
发布基站 OTA 任务
topic: /estation/90A9F7300000/ota
Payload:
{
  Firmware: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=",
  MD5: "19ec7e38acb276f6b71386fd9be21e9f",
  Type: 0,
  Version: "1.6.5"
}
```

3 参考

3.1 数据定义 (Java 版本)

eStationInfor: 用于定义 eStation 的配置、状态、参数等信息。其中 ID、MAC、AppVersion、DummyVersion、TotalCount、SendCount 是只读的。它的定义如下:

```
/**
 * Client ID - readonly
 */
public String ID = "";
/**
 * MAC address - readonly
 */
public String MAC = "";
/**
 * Alias, only for display
 */
public String Alias = "";
/**
 * Client type
 */
public int ClientType;
/**
 * Server IP + port address
 * 默认值为 192.168.1.192:9080
 */
public String ServerAddress = "";

/**
 * Connection parameters with connection type
 * 0 - NULL
 * 1 - Certial name, password
 * 2 - User ID, password
 */
public String Parameters[];
/**
 * Local IP address, keep empty means auto obtain an IP address
 * If local IP address is empty, will ignore subnet mask and gateway
 */
public String LocalIP = "";
/**
 * Subnet mask, will use empty when LocalIP is empty
 */
public String SubnetMask = "";
/**
 * Gateway, will use empty when LocalIP is empty
 */
public String Gateway = "";
/**
 * Heartbeat, less than 15 is auto reset to 15
 */
```

```

*/
public int Heartbeat;
/**
    * [Readonly]App version
*/
public String AppVersion = "";
/**
    * [Readonly]Task in working channel
*/
public int TotalCount;
/**
    * [Readonly]Task in cache
*/
public int SendCount;

```

TaskData: 用于定义 PTL(灯条)的任务数据信息。它的定义如下:

```

/**
    * LED light flashing time, =Time*5 seconds
*/
public int Time;
/**
    * Task items
*/
public List<TaskItemData> Items = null;

```

TaskItemData: 用于定义 PTL(灯条)的任务数据信息。它的定义如下:

```

/**
    * Tag ID
*/
public String TagID = "";
/**
    * Beep
*/
public Boolean Beep = false;
/**
    * LED light flashing times
*/
public Boolean Flashing = false;
/**
    * Light color
    * LightColorProtocol->
    *     public boolean R = false;
    *     public boolean G = false;
    *     public boolean B = false;

```

```
*/  
public List<LightColorProtocol> Colors = null;
```

GroupData: 用于定义群控的任务数据信息。它的定义如下:

```
/**  
 * LED light flashing times  
 */  
public int Times;  
/**  
 * Red LED light  
 */  
public boolean R = false;  
/**  
 * Green LED light  
 */  
public boolean G = false;  
/**  
 * Blue LED light  
 */  
public boolean B = false;  
/**  
 * Beep  
 */  
public boolean Beep = false;
```

TaskResult: 用于定义一组 PTL (灯条)通信结果的数据信息。它的定义如下:

```
/**  
 * AP ID  
 */  
public String ID = "";  
/**  
 * Total count  
 */  
public int TotalCount = -1;  
/**  
 * Send count  
 */  
public int SendCount = -1;  
/**  
 * Task item results  
 */  
public List<TaskItemResult> Results;
```


TaskItemResult: 用于定义单个 PTL (灯条)通信结果的数据信息。它的定义如下:

```
/**
 * Tag ID
 */
public String TagID = "";
/**
 * Version
 */
public String Version = "";
/**
 * 0xFD 按键返回, 0xFE 通信返回, 0xFF 心跳返回
 */
public int ResultType = 0;
/**
 *
 */
public int RfPowerSend = 0;
/**
 *
 */
public int RfPowerRecv = 0;
/**
 * 电池电量, 电压 V, 0 表示无数据, 需要除以 10
 */
public int Battery = -1;
/**
 * LED 灯的当前状态 [R, G, B]
 */
public List<LightColorProtocol> Colors = null;
```

RGB: 用于定义 PTL(灯条)的单个 RGB 灯的数据信息。它的定义如下:

```
/**
 * Red LED light
 */
public boolean R = false;
/**
 * Green LED light
 */
public boolean G = false;
/**
 * Blue LED light
 */
public boolean B = false;
```

3.2 数据定义（C#版本）

eStationInfor：用于定义 eStation 的配置、状态、参数等信息。其中 ID、MAC、AppVersion、DummyVersion、TotalCount、SendCount 是只读的。它的定义如下：

```
public class eStationInfor
{
    /// <summary>
    /// Client ID - readonly
    /// </summary>
    public string ID { get; set; }
    /// <summary>
    /// MAC address - readonly
    /// </summary>
    public string MAC { get; set; }
    /// <summary>
    /// Alias, only for display
    /// </summary>
    public string Alias { get; set; }
    /// <summary>
    /// Client type
    /// </summary>
    public int ClientType { get; set; }
    /// <summary>
    /// Server IP + port address
    /// </summary>
    public string ServerAddress { get; set; }
    /// <summary>
    /// Connection parameters with connection type
    /// 0 - NULL
    /// 1 - Certial name, password
    /// 2 - User ID, password
    /// </summary>
    public string[] Parameters { get; set; }
    /// <summary>
    /// Local IP address, keep empty means auto obtain an IP address
    /// If local IP address is empty, will ignore subnet mask and
gateway
    /// </summary>
    public string LocalIP { get; set; }
    /// <summary>
    /// Subnet mask, will use empty when LocalIP is empty
    /// </summary>
    public string SubnetMask { get; set; }
    /// <summary>
    /// Gateway, will use empty when LocalIP is empty
    /// </summary>
    public string Gateway { get; set; }
    /// <summary>
    /// Heartbeat, less than 15 is auto reset to 15
    /// </summary>
    public int Heartbeat { get; set; } = 15;
    /// <summary>
    /// [Readonly]App version
    /// </summary>
}
```

```

        public string AppVersion { get; set; }
        /// <summary>
        /// [Readonly]Task in working channel
        /// </summary>
        public int TotalCount { get; set; } = 0;
        /// <summary>
        /// [Readonly]Task in cache
        /// </summary>
        public int SendCount { get; set; } = 0;
    }

```

TaskData: 用于定义 PTL(灯条)的任务数据信息。它的定义如下:

```

/// <summary>
/// Task data entity
/// </summary>
public class TaskData
{
    /// <summary>
    /// LED light flashing time, =Time*5seconds
    /// </summary>
    public int Time { get; set; } = 1;
    /// <summary>
    /// Task items
    /// </summary>
    public TaskItemData[] Items { get; set; } =
Array.Empty<TaskItemData>();
}

```

TaskItemData: 用于定义 PTL(灯条)的任务数据信息。它的定义如下:

```

/// <summary>
/// Task item data entity
/// </summary>
public class TaskItemData
{
    /// <summary>
    /// Tag ID
    /// </summary>
    public string TagID { get; set; } = "";
    /// <summary>
    /// Beep
    /// </summary>
    public bool Beep { get; set; } = false;
    /// <summary>
    /// Light color
    /// </summary>
    public RGB[] Colors { get; set; } = Array.Empty<RGB>();
    /// <summary>
    /// LED light flashing times
    /// </summary>
    public bool? Flashing { get; set; } = null;
}

```

GroupData: 用于定义群控的任务数据信息。它的定义如下:

```

/// <summary>
/// Task Data
/// </summary>
public class GroupData
{
    /// <summary>
    /// Red LED light
    /// </summary>
    public bool R { get; set; } = false;
    /// <summary>

```

```

    /// Green LED light
    /// </summary>
    public bool G { get; set; } = false;
    /// <summary>
    /// Blue LED light
    /// </summary>
    public bool B { get; set; } = false;
    /// <summary>
    /// Beep
    /// </summary>
    public bool Beep { get; set; } = false;
    /// <summary>
    /// LED light flashing times
    /// </summary>
    public int Times { get; set; } = 0;
}

```

TaskResult: 用于定义一组 PTL (灯条)通信结果的数据信息。它的定义如下:

```

    /// <summary>
    /// Task result entity
    /// </summary>
    public class TaskResult
    {
        /// <summary>
        /// AP ID
        /// </summary>
        public string ID { get; set; } = string.Empty;

        /// <summary>
        /// Total count
        /// </summary>
        public int TotalCount { get; set; }

        /// <summary>
        /// Send count
        /// </summary>
        public int SendCount { get; set; }

        /// <summary>
        /// Task item results
        /// </summary>
        public TaskItemResult[] Results { get; set; } =
        Array.Empty<TaskItemResult>();
    }

```

TaskItemResult: 用于定义单个 PTL (灯条)通信结果的数据信息。它的定义如下:

```

    /// <summary>
    /// Task item result
    /// </summary>
    public class TaskItemResult
    {
        /// <summary>
        /// Tag ID
        /// </summary>
        public string TagID { get; set; } = string.Empty;
        /// <summary>
        /// Version
        /// </summary>
        public string Version { get; set; } = string.Empty;
        /// <summary>
        /// Screen
        /// </summary>
        public int ResultType { get; set; } = 0;
    }

```

```

    /// <summary>
    /// RSSI
    /// </summary>
    public int RfPowerSend { get; set; } = -256;
    /// <summary>
    /// RSSI
    /// </summary>
    public int RfPowerRecv { get; set; } = -256;
    /// <summary>
    /// Battery Power
    /// </summary>
    public int Battery { get; set; } = 0;
    /// <summary>
    /// Battery Power
    /// </summary>
    public RGB[] Colors { get; set; } = Array.Empty<RGB>();
}

```

RGB: 用于定义 PTL(灯条)的单个 RGB 灯的数据信息。它的定义如下:

```

    /// <summary>
    /// RGB
    /// </summary>
    public class RGB
    {
        /// <summary>
        /// Red LED light
        /// </summary>
        public bool R { get; set; } = false;
        /// <summary>
        /// Green LED light
        /// </summary>
        public bool G { get; set; } = false;
        /// <summary>
        /// Blue LED light
        /// </summary>
        public bool B { get; set; } = false;
        /// <summary>
        /// Default constructor
        /// </summary>
        public RGB() { }
        /// <summary>
        /// Constructor
        /// </summary>
        /// <param name="r">Red</param>
        /// <param name="g">Green</param>
        /// <param name="b">Blue</param>
        public RGB(bool r, bool g, bool b)
        {
            R = r;
            G = g;
            B = b;
        }
    }
}

```

3.3 电量描述

灯条的电压值与电量等级换算关系如下：

电压值	换算电量等级
>=3.0	100%
>=2.9	90%
>=2.8	80%
>=2.7	60%
>=2.6	30%
>=2.5	10%
<2.5	0%（需要尽快更换电池）

3.4 颜色描述

灯条搭载有 RGB 三色灯，可以通过单色/组合色达到如下灯色：

R	G	B	颜色
√			红
	√		绿
		√	蓝
√	√		黄
	√	√	青
√		√	紫
√	√	√	白